EXISTS is a SQL operator used to check whether a subquery returns at least one row.

- If the subquery returns one or more rows → EXISTS returns TRUE.
- If the subquery returns no rows → EXISTS returns FALSE.

It is commonly used with correlated subqueries to check whether a related record exists in another table.

1. Basic syntax

```
SELECT columns

FROM table1

WHERE EXISTS (

    SELECT 1

    FROM table2

    WHERE condition

);
```

The important thing is that EXISTS checks whether rows exist. It does not care about the actual values returned by the subquery.

For example, these are equivalent for an EXISTS check:

```
WHERE EXISTS (SELECT 1 FROM orders ...)

WHERE EXISTS (SELECT * FROM orders ...)
```

Using SELECT 1 is a common convention because we only care about existence.

2. Practical example

Consider two tables.

Customers

customer_id	customer_name
1	Amit
2	Rahul
3	Priya
4	Neha

Orders

order_id	customer_id	amount
101	1	500
102	1	700
103	2	300
104	3	900

Requirement: Find customers who have placed at least one order.

```
SELECT c.customer_id,
       c.customer_name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

Result:

customer_id	customer_name
1	Amit
2	Rahul
3	Priya

Neha is excluded because she has no matching order.

3. How EXISTS works internally

Let's understand what happens for each customer.

For Amit:

```
SELECT 1
```

```
FROM orders o

WHERE o.customer_id = 1;
```

The result contains matching rows, so EXISTS returns TRUE.

For Rahul:

```
SELECT 1

FROM orders o

WHERE o.customer_id = 2;
```

A matching row exists, so TRUE.

For Neha:

```
SELECT 1

FROM orders o

WHERE o.customer_id = 4;
```

No matching rows exist, so FALSE.

EXISTS evaluation

Customer	Matching order?	EXISTS
Amit	Yes	TRUE
Rahul	Yes	TRUE
Priya	Yes	TRUE

Neha	No	FALSE

Only customers for whom EXISTS is TRUE are returned.

Important: The database can stop searching once it finds a qualifying row. It doesn't need to count all matching orders.

4. NOT EXISTS

NOT EXISTS does the opposite.

Requirement: Find customers who have never placed an order.

```
SELECT c.customer_id,
       c.customer_name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

Result:

customer_id	customer_name
4	Neha

This is called an anti-existence check or anti-join pattern.

5. EXISTS vs IN vs JOIN

These three approaches can sometimes solve the same problem, but their behavior differs.

A. Using EXISTS

```
SELECT c.*

FROM customers c

WHERE EXISTS (

    SELECT 1

    FROM orders o

    WHERE o.customer_id = c.customer_id

);
```

B. Using IN

```
SELECT *

FROM customers

WHERE customer_id IN (

    SELECT customer_id

    FROM orders

);
```

C. Using JOIN

```
SELECT DISTINCT c.*

FROM customers c

JOIN orders o

    ON o.customer_id = c.customer_id;
```

Comparison:

Feature	EXISTS	IN	JOIN
Checks existence	Yes	Yes	Not inherently
Duplicate matching rows	Doesn't duplicate outer rows	Doesn't duplicate outer rows	Can duplicate rows
Returns columns from both tables	No	No	Yes
Common use	Existence checks	Membership checks	Combining related data

Important: NOT EXISTS vs NOT IN

Be careful when using `NOT IN`.

If the subquery returns even one `NULL`, `NOT IN` can produce unexpected results because of SQL's three-valued logic.

For example:

```
SELECT *

FROM customers

WHERE customer_id NOT IN (

    SELECT customer_id

    FROM orders
```

```
);
```

If `orders.customer_id` contains NULL, this query may return no rows.

`NOT EXISTS` avoids this particular problem:

```
SELECT c.*
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

6. More practical interview examples

Example 1: Find employees who have at least one project.

```
SELECT e.*
FROM employees e
WHERE EXISTS (
    SELECT 1
    FROM employee_projects ep
    WHERE ep.employee_id = e.employee_id
);
```

Example 2: Find departments that have no employees.

```
SELECT d.*
FROM departments d
```

```
WHERE NOT EXISTS (
    SELECT 1
    FROM employees e
    WHERE e.department_id = d.department_id
);
```

Example 3: Find customers who have placed an order greater than ₹10,000.

```
SELECT c.*
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
      AND o.amount > 10000
);
```

Example 4: Find employees who have at least one colleague earning more than them in the same department.

```
SELECT e.*
FROM employees e
WHERE EXISTS (
    SELECT 1
    FROM employees e2
    WHERE e2.department_id = e.department_id
```

```
      AND e2.salary > e.salary
);
```

This last example is particularly interesting because the same table is used in both the outer and inner queries.

7. Test your understanding

Suppose a customer has placed 5 orders.

What will this query return for that customer?

```
SELECT c.customer_name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

5 rows, one for each order

1 row, one for the customer

No rows

Check answer

Remember: EXISTS answers the question "Does at least one matching row exist?" It is one of the most useful SQL patterns for correlated subqueries, semi-joins and anti-joins.